
Q1.

An instruction for the ATMega32u2 is shown in Table 1.

Table 1: BRNE — branch if not equal — opcode specification

Syntax	Operands	Operation
`brne k`	$-64 \leq k \leq 63$	Result of CP, CPI, SUB, or SUBI $\neq 0$?
Control registers	Opcode	Flags
PC $\leftarrow$ PC + k + 1 (if Z = 0) PC $\leftarrow$ PC + 1 (if Z = 1)	1111 01kk kkkk k001	None

For the instruction 'branch if not equal':

(a) State what type/class of instruction it is.

(b) What is the binary sequence encoded by the hexadecimal `0xF4A1`?

(c) Assuming that k is represented as a 7-bit two's complement integer, what is the value of k if the specific opcode sequence is `0xF4A1` ?

Another AVR instruction is shown in Table 2

Table 2: CPI — Compare immediate — opcode. Note: d has an offset of +16

Syntax	Operands	Operation
`cpi RD,K`	$16 \leq d \leq 31$, $0 \leq K \leq 255$	Rd - K
Control registers	Opcode	Flags
PC $\leftarrow$ PC + 1	0011 KKKK dddd KKKK	H,S,V,N,Z,C

(d) What does immediate addressing mean?

(e) What is the value of `PC` after the following two instructions?

PC	opcode (hex)	instruction
...		
0x0008:	3c 20	cpi rXX 0xXX;
0x0009:	f4 a1	brne, 20
...		

Register	Value
...	
R16	01
R17	00
R18	c0
R19	1a
R20	3c
R21	c2
R22	00
R23	f0
...	

Table 3: `call` instruction

Syntax	Operands	Operation
`call k`	$0 \leq k \leq 64K$	STACK $\leftarrow$ PC + 2
Control registers	Opcode	Flags
PC $\leftarrow$ k SP $\leftarrow$ SP - 2	1001 010k kkkk 111k kkkk kkkk kkkk kkkk	None

Q2.

Consider the call instruction, shown in Table 3

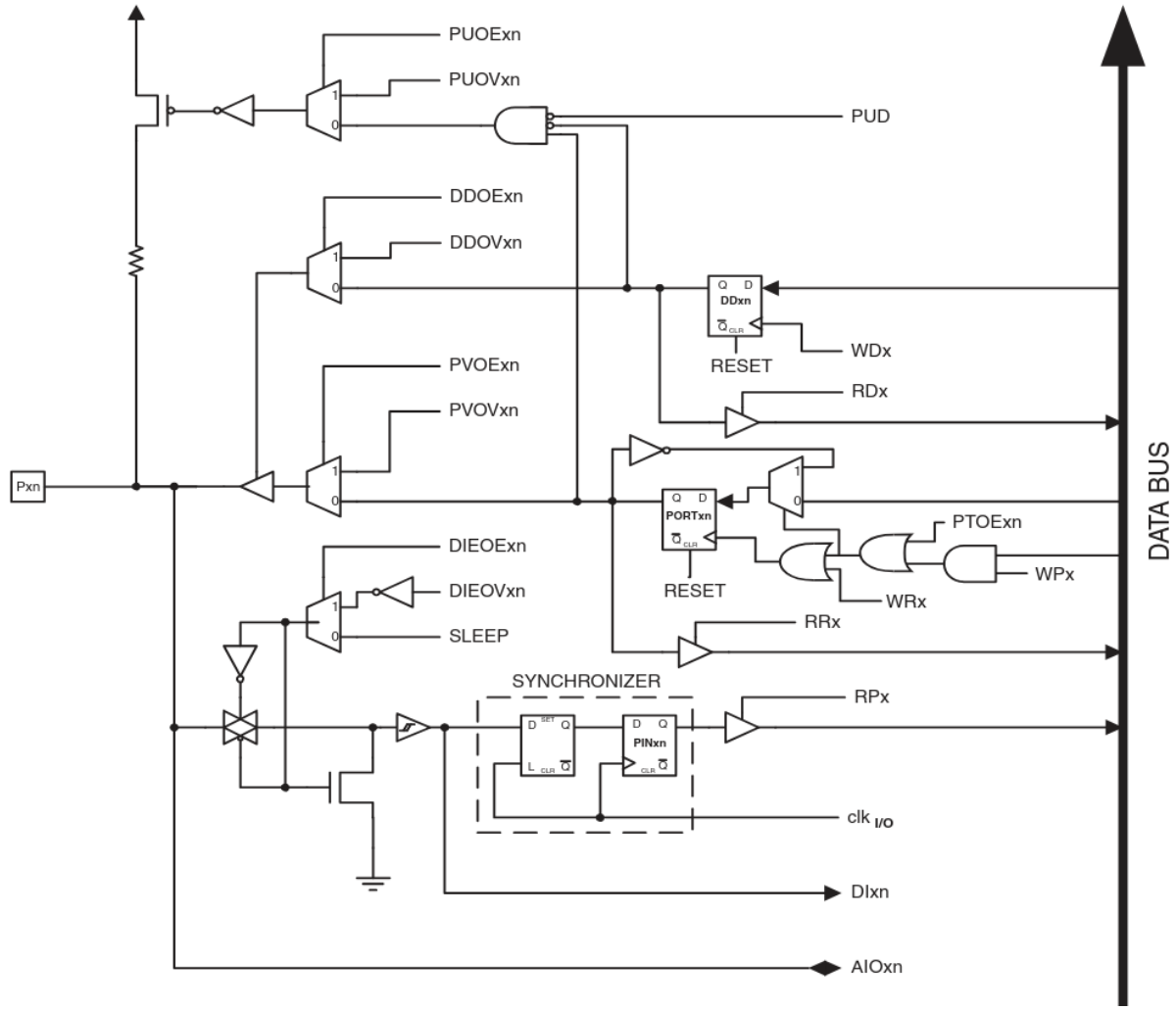
(a) Explain what function changing the value of `PC` has.

(b) Explain why a call instruction is necessary, as opposed to a `brxx` branching instruction.

(c) Explain why PC + 2 is stored on the stack, and why the SP is decremented by 2.

Q3.

Consider the alternate port function diagram shown in Figure 1.



PUOExn:	Pxn PULL-UP OVERRIDE ENABLE	PUD:	PULLUP DISABLE
PUOVxn:	Pxn PULL-UP OVERRIDE VALUE	WDx:	WRITE DDRx
DDOExn:	Pxn DATA DIRECTION OVERRIDE ENABLE	RDx:	READ DDRx
DDOVxn:	Pxn DATA DIRECTION OVERRIDE VALUE	RRx:	READ PORTx REGISTER
PVOExn:	Pxn PORT VALUE OVERRIDE ENABLE	WRx:	WRITE PORTx
PVOVxn:	Pxn PORT VALUE OVERRIDE VALUE	RPx:	READ PORTx PIN
DIEOExn:	Pxn DIGITAL INPUT-ENABLE OVERRIDE ENABLE	WPx:	WRITE PINx
DIEOVxn:	Pxn DIGITAL INPUT-ENABLE OVERRIDE VALUE	$clk_{I/O}$:	I/O CLOCK
SLEEP:	SLEEP CONTROL	DIxn:	DIGITAL INPUT PIN n ON PORTx
PTOExn:	Pxn, PORT TOGGLE OVERRIDE ENABLE	AIOxn:	ANALOG INPUT/OUTPUT PIN n ON PORTx

Figure 1: Diagram of the internal GPIO circuitry of the ATMega32u2, including alternate function circuitry.

(a) For the pin to be in output mode, explain what the tristate buffer closest to the pin will need to do, and what signal it needs on the `enable` connection to do this.

(b) Determine what value `PVOExn` will need to be in to allow the signal from `PORTxn` to be selected in the multiplexer connected to the above tristate buffer.

(c) Complete the remaining values required for the signals below to place the pin into standard GPIO operating mode, with a `HIGH` output.

Signal	State	Signal	State
`PUOExn`		`PUOVxn`	
`DDOVxn`		`DDOExn`	
`DDxn`		`PVOExn`	
`PVOVxn`		`PORTxn`	
(others)			0

(d) Determine what values are required for signals in the below Table to place the pin into alternate function mode, configured as an input with the pullup enabled.

Signal	State	Signal	State
`PUOExn`		`PUOVxn`	
`DDOVxn`		`DDOExn`	
`DDxn`		`PVOExn`	
`PVOVxn`		`PORTxn`	
(others)			0

END OF QUESTIONS